

BAB II

TINJAUAN PUSTAKA

2.1 Tinjauan Penelitian Terdahulu

Tinjauan Tinjauan penelitian terdahulu digunakan sebagai dasar untuk mengetahui perkembangan kajian, metode yang digunakan, serta celah yang masih dapat dikembangkan. Pada topik pengelolaan aduan, beberapa studi menunjukkan bahwa proses manual sering kali kurang efisien sehingga dibutuhkan pendekatan otomatis berbasis kecerdasan buatan, klasifikasi teks, dan mekanisme prioritas.

Ananto, dkk. (2019) mengklasifikasikan kategori pengaduan masyarakat pada kanal LAPOR! menggunakan Artificial Neural Network (ANN) yang dikombinasikan dengan SMOTE untuk mengatasi ketidakseimbangan data dan metode pemilihan kata kunci untuk seleksi fitur, menghasilkan tingkat akurasi dari 428 kata kunci yang berpengaruh. Studi ini terbatas pada satu kanal resmi pemerintah dan mengandalkan model yang harus dilatih ulang dengan data yang sudah dikategorikan. Sistem yang dikembangkan dalam Tugas Akhir ini berbeda karena mengambil aduan dari beberapa kanal media sosial sekaligus (Facebook dan Instagram) serta memanfaatkan model AI yang sudah dilatih sebelumnya (Gemma) sehingga tidak memerlukan pelatihan ulang, sekaligus menangani istilah dialek lokal yang tidak dibahas pada studi tersebut.

Zarly, dkk. (2026) membangun sistem klasifikasi pengaduan otomatis berbasis web menggunakan teknik klasifikasi otomatis di Dinas Kependudukan dan Pencatatan Sipil Kabupaten Pesisir Selatan, dengan integrasi Laravel dan Flask API, dievaluasi menggunakan tingkat akurasi. Sistem tersebut spesifik untuk satu instansi dan satu algoritma metode machine learning lama yang membutuhkan pelatihan model. Sistem yang dikembangkan dalam Tugas Akhir ini mencakup klasifikasi otomatis ke 32 OPD sekaligus tanpa proses pelatihan model karena berbasis LLM generatif.

Prayogo, dkk. (2026) meneliti klasifikasi validitas pengaduan masyarakat (klasifikasi dua kategori valid/tidak valid) menggunakan pendekatan deep learning terhadap 2.000 data aduan, dilatarbelakangi temuan bahwa 13% instansi pemerintah daerah tidak memenuhi SLA 100% pada tahun 2023. Studi ini berhenti pada

klasifikasi dua kategori validitas aduan tanpa cara kerja tindak lanjut. Sistem yang dikembangkan dalam Tugas Akhir ini melangkah lebih jauh dengan filter spam dua tahap, klasifikasi banyak kategori (kategori, sub-kategori, OPD), serta cara kerja tiket dengan eskalasi otomatis berbasis SLA 24 jam.

Mazia, dkk. (2021) merancang sistem informasi helpdesk ticketing berbasis web untuk kebutuhan internal PT Mitra Tiga Berlian, di mana tiket dibuat manual oleh staf saat terjadi kendala TI. Sistem yang dikembangkan dalam Tugas Akhir ini berbeda karena tiket terbentuk otomatis dari hasil penyaringan dan klasifikasi AI atas aduan publik yang diambil langsung dari media sosial, dilengkapi SLA 24 jam dan eskalasi prioritas otomatis—mekanisme yang tidak ada pada sistem helpdesk internal tersebut.

Berdasarkan keempat studi tersebut, klasifikasi pengaduan berbasis AI, sistem ticketing, dan pengukuran validitas/SLA masing-masing telah diteliti secara terpisah. Tugas Akhir ini mengintegrasikan ketiganya sekaligus—pengumpulan otomatis multi-kanal, klasifikasi berbasis LLM generatif tanpa pelatihan ulang dengan dukungan dialek lokal, serta manajemen tiket dengan SLA dan eskalasi otomatis dalam satu sistem yang belum ditemukan pada studi-studi sebelumnya.

2.2 Tinjauan Pustaka Berkaitan dengan Judul

2.2.1 Sistem Informasi

Sistem informasi merupakan kombinasi dari komponen-komponen yang terdiri atas manusia, teknologi informasi, dan prosedur kerja yang memproses, menyimpan, menganalisis, dan menyebarkan informasi untuk tujuan tertentu dalam suatu organisasi (Kadir, 2019). Sistem informasi berperan sebagai pendukung utama dalam proses pengambilan keputusan, koordinasi, dan pengendalian dalam organisasi dengan menyediakan informasi yang akurat, tepat waktu, dan relevan. Dalam konteks pelayanan publik, sistem informasi membantu instansi pemerintah untuk mengelola data masyarakat, memproses laporan atau keluhan, serta menyajikan informasi yang dibutuhkan untuk evaluasi kinerja.

2.2.2 Metode Agile

Metode Agile adalah pendekatan pengembangan perangkat lunak yang menekankan pada fleksibilitas, iterasi cepat, dan kolaborasi dalam proses

pengembangan (Pressman, 2021). Berbeda dengan metode tradisional yang kaku dan dilakukan secara berurutan, Agile memungkinkan pengembang untuk merespons perubahan kebutuhan dengan cepat melalui siklus pengembangan yang pendek dan berulang. Metode ini pertama kali diperkenalkan melalui Agile Manifesto pada tahun 2001 dan sejak itu menjadi standar industri dalam pengembangan software modern.

Metode Agile memiliki empat nilai inti, individu dan interaksi lebih penting daripada proses dan tools, software yang berfungsi lebih penting daripada dokumentasi yang lengkap, kolaborasi dengan pengguna lebih penting daripada negosiasi kontrak, dan merespons perubahan lebih penting daripada mengikuti rencana awal. Nilai-nilai ini memandu pengembang untuk fokus pada hasil yang dapat digunakan dan fleksibel terhadap perubahan.

Secara umum, penerapan metode Agile melibatkan empat tahapan utama, yaitu pengumpulan kebutuhan (requirements), perancangan (design), pengembangan (development), dan pengujian (testing). Setiap tahapan dapat diiterasi sesuai kebutuhan untuk meningkatkan kualitas hasil akhir. Metode ini sangat cocok untuk pengembangan sistem berbasis kecerdasan buatan yang membutuhkan iterasi dan penyesuaian berkelanjutan, karena proses tuning model AI dan testing akurasi memerlukan siklus pengembangan yang fleksibel. Selain itu, metode ini memungkinkan pengembangan bertahap dimana setiap fitur dapat diuji dan diperbaiki sebelum melanjutkan ke fitur berikutnya, sehingga meminimalkan risiko kegagalan sistem secara keseluruhan.

2.2.3 Aplikasi Berbasis Web

Aplikasi berbasis web adalah perangkat lunak yang dijalankan pada server dan diakses melalui browser web, tanpa memerlukan instalasi pada perangkat klien (Rosa & Shalahuddin, 2018). Berbeda dengan aplikasi desktop tradisional yang harus diinstal di setiap perangkat pengguna, aplikasi web hanya memerlukan browser dan koneksi internet untuk diakses. Hal ini memudahkan proses update karena perubahan cukup dilakukan di server saja, tanpa perlu mengupdate setiap perangkat klien secara manual.

Aplikasi web juga memungkinkan akses bersamaan dari berbagai perangkat dan lokasi, sehingga cocok untuk sistem yang melayani banyak pengguna seperti

dashboard pengelolaan aduan publik. Dalam konteks KMC, pendekatan berbasis web memungkinkan petugas dari berbagai OPD untuk memantau dan menangani aduan tanpa terikat pada satu perangkat atau lokasi kantor tertentu.

2.2.4 Manajemen Aduan

Manajemen aduan adalah proses sistematis untuk menerima, mencatat, mengklasifikasikan, menindaklanjuti, dan menyelesaikan keluhan atau laporan dari masyarakat (ISO 10002:2018). Sistem manajemen aduan yang efektif harus mampu menangani jumlah besar, memberikan transparansi kepada pelapor, serta memastikan tanggung jawab penanganan oleh instansi terkait. Standar ISO 10002:2018 menetapkan pedoman untuk merancang dan menerapkan proses penanganan keluhan yang efektif dan efisien, termasuk prinsip-prinsip seperti kemudahan akses, kecepatan respon, objektivitas, kerahasiaan, dan pendekatan berorientasi pelanggan..

2.2.5 Multi-Channel

Multi-channel adalah pendekatan dalam penyediaan layanan yang memungkinkan pengguna untuk berinteraksi dengan organisasi melalui berbagai saluran komunikasi yang terintegrasi (Rangkuti, 2019). Dalam konteks pengelolaan aduan publik, strategi multi-channel berarti masyarakat tidak harus terbatas pada satu saluran saja—mereka dapat memilih menyampaikan laporan melalui Facebook, Instagram, website resmi, atau saluran lain sesuai preferensi masing-masing.

Pendekatan ini penting karena tidak semua kelompok masyarakat menggunakan platform yang sama. Generasi muda cenderung aktif di media sosial, sementara kelompok lain mungkin lebih nyaman menggunakan email atau telepon. Dengan mengintegrasikan berbagai saluran ke dalam satu sistem terpusat, KMC dapat memastikan bahwa setiap aduan—dari mana pun asalnya—tercatat, terpantau, dan tertangani dengan standar yang sama..

2.2.6 Notifikasi Real-Time

Notifikasi real-time adalah mekanisme penyampaian informasi secara instan kepada pengguna pada saat terjadinya suatu kejadian atau perubahan status dalam sistem (Nugroho & Purnama, 2020). Sistem notifikasi real-time memungkinkan pengguna untuk segera mengetahui informasi penting tanpa harus melakukan

pemeriksaan manual secara berkala, sehingga meningkatkan responsivitas dalam pengambilan keputusan. Teknologi yang umum digunakan untuk notifikasi real-time meliputi WebSocket, Server-Sent Events (SSE), atau mekanisme polling pada interval pendek (teknologi notifikasi otomatis).

2.2.7 Artificial Intelligence (AI)

Artificial Intelligence (AI) merupakan cabang ilmu komputer yang berfokus pada pengembangan sistem yang dapat meniru kemampuan berpikir manusia dalam menyelesaikan suatu permasalahan (Russell & Norvig, 2020). AI digunakan untuk membantu sistem dalam memahami data, menganalisis informasi, serta menghasilkan rekomendasi yang dapat mendukung proses pengambilan keputusan secara lebih cepat dan terarah. Dalam perkembangannya, AI telah diterapkan pada berbagai bidang seperti pemrosesan bahasa alami, pengenalan pola, klasifikasi teks, hingga sistem rekomendasi.

2.2.8 Klasifikasi Aduan

Klasifikasi aduan adalah proses pengelompokan aduan ke dalam kategori tertentu berdasarkan isi dan konteks pesan yang disampaikan (Wicaksono & Purwarianti, 2020). Klasifikasi teks secara otomatis memanfaatkan teknik pemrosesan bahasa alami untuk menganalisis dan mengkategorikan dokumen teks ke dalam kelas-kelas yang telah ditentukan sebelumnya, sehingga mempercepat proses penanganan dan meningkatkan akurasi distribusi tugas. Teknik klasifikasi yang umum digunakan meliputi algoritma Naive Bayes, Support Vector Machine (SVM), Random Forest, dan pendekatan berbasis deep learning seperti neural network atau transformer.

2.2.9 Deteksi Duplikasi Aduan

Deteksi duplikasi aduan adalah proses identifikasi terhadap dua atau lebih aduan yang memiliki kesamaan isi, topik, atau merujuk pada permasalahan yang sama (Christanto & Tjahyana, 2021). Duplikasi sering terjadi ketika satu permasalahan dilaporkan oleh beberapa warga melalui akun berbeda, atau ketika satu pengguna mengirimkan aduan serupa melalui kanal yang berbeda dalam waktu berdekatan.

Sebagai contoh, jika tiga warga melaporkan masalah "jalan berlubang di Jl. Sudirman" melalui Facebook dan Instagram dalam satu hari, sistem perlu mengenali bahwa ketiga laporan tersebut merujuk pada permasalahan yang sama

agar tidak membuat tiga tiket terpisah. Deteksi duplikasi juga membantu mengidentifikasi tingkat urgensi—jika banyak warga melaporkan masalah yang sama secara bersamaan, hal ini mengindikasikan bahwa permasalahan tersebut perlu diprioritaskan.

Metode deteksi duplikasi dapat menggunakan pendekatan kemiripan teks berbasis metode penghitungan kemiripan teks yang membandingkan konten aduan dengan laporan yang masuk dalam rentang waktu tertentu, dan bisa juga menggunakan pendekatan berbasis Large Language Model dengan membandingkan kesamaan masalah, lokasi, dan objek yang dilaporkan pada aduan baru terhadap aduan sebelumnya.

2.2.10 Prioritas Eskalasi Aduan

Prioritas eskalasi aduan adalah mekanisme penentuan tingkat urgensi suatu aduan berdasarkan kriteria tertentu, yang menentukan seberapa cepat aduan tersebut harus ditangani (Prasetyo & Suyanto, 2019). Eskalasi merupakan proses peningkatan level penanganan ketika aduan tidak ditindaklanjuti dalam batas waktu yang ditentukan, memastikan bahwa setiap laporan masyarakat mendapatkan perhatian yang semestinya.

Tidak semua aduan memiliki tingkat urgensi yang sama—laporan tentang jalan rusak yang menghambat akses umum tentu lebih mendesak dibanding keluhan administrasi rutin. Sistem eskalasi otomatis bekerja dengan memantau waktu penanganan setiap tiket dan secara otomatis menaikkan prioritas jika batas waktu terlampaui, sehingga aduan tidak terabaikan meskipun petugas sedang menangani volume tinggi. Mekanisme ini juga membantu alokasi sumber daya yang lebih tepat, karena petugas dapat fokus menangani aduan dengan prioritas tertinggi terlebih dahulu.

2.2.11 Service Level Agreement (SLA)

Service Level Agreement (SLA) adalah komitmen formal antara penyedia layanan dan pengguna mengenai standar kualitas layanan yang dijanjikan, termasuk batas waktu penyelesaian suatu permintaan atau keluhan (Hidayat & Muttaqin, 2020). SLA berfungsi sebagai acuan untuk mengukur kinerja pelayanan dan memastikan akuntabilitas pihak penyedia layanan dalam memenuhi ekspektasi pengguna.

Dalam konteks pelayanan publik, SLA membantu instansi pemerintah untuk menetapkan standar waktu respons yang jelas dan terukur, sehingga masyarakat memiliki kepastian mengenai waktu penyelesaian aduan mereka.

2.2.12 Dashboard

Dashboard adalah tampilan visual dari data-data penting yang diperlukan untuk mencapai suatu tujuan, yang disusun dan dipadukan dalam satu tampilan sehingga dapat dilihat secara menyeluruh (Irawan & Simargolang, 2019). Dashboard memungkinkan pengguna untuk memantau kondisi terkini, mengidentifikasi tren, dan mengambil keputusan secara cepat berdasarkan data yang disajikan secara ringkas dan informatif.

Dashboard yang efektif dirancang untuk menyajikan informasi penting dalam satu halaman tanpa perlu berpindah-pindah menu, menggunakan visualisasi seperti grafik dan indikator status yang mudah dipahami sekilas, serta mendukung pembaruan data secara berkala atau real-time. Dalam konteks pengelolaan aduan, dashboard membantu manajemen untuk melihat seberapa banyak aduan yang masuk hari ini, berapa yang sudah ditangani, dan mana yang mendekati batas waktu SLA. Semua informasi ini tersedia dalam satu tampilan untuk mendukung pengambilan keputusan yang cepat.

2.2.13 Sistem Ticketing

Sistem ticketing adalah cara kerja pengelolaan permintaan layanan atau laporan yang menggunakan tiket sebagai unit pencatatan, pelacakan, dan penyelesaian setiap kasus (Mazia dkk., 2021). Setiap tiket menggambarkan satu aduan atau permasalahan yang memiliki siklus hidup mulai dari pembuatan, penugasan, penanganan, hingga penyelesaian. Dengan pendekatan ticketing, setiap aduan tercatat dengan nomor unik sehingga mudah dilacak status penanganannya secara transparan, mendukung cara kerja disposisi ke pihak yang bertanggung jawab, dan menyediakan riwayat penanganan yang berguna untuk evaluasi kinerja di masa mendatang.

2.2.14 Organisasi Perangkat Daerah (OPD)

Organisasi Perangkat Daerah (OPD) adalah instansi pemerintah daerah yang memiliki tugas dan fungsi tertentu dalam penyelenggaraan pemerintahan sesuai

dengan Undang-Undang No. 23 Tahun 2014 tentang Pemerintahan Daerah. OPD berperan sebagai pelaksana urusan pemerintahan di tingkat daerah, baik yang bersifat wajib maupun pilihan. Dalam konteks pengelolaan aduan masyarakat, OPD merupakan pihak yang bertanggung jawab untuk menangani permasalahan sesuai dengan bidang tugasnya, seperti Dinas Perhubungan untuk masalah transportasi, Dinas Pekerjaan Umum untuk masalah infrastruktur, atau Dinas Kesehatan untuk masalah layanan kesehatan.

2.2.15 Natural Language Processing (NLP)

Natural Language Processing (NLP) adalah cabang kecerdasan buatan yang fokus pada interaksi antara komputer dan bahasa manusia (Jurafsky & Martin, 2023). NLP memungkinkan mesin untuk memahami, menginterpretasikan, dan menghasilkan bahasa natural sehingga dapat digunakan untuk berbagai tugas pengolahan teks seperti klasifikasi, ekstraksi informasi, dan analisis sentimen. Teknik NLP mencakup tokenisasi (pemecahan teks menjadi unit-unit kecil), part-of-speech tagging (penandaan jenis kata), named entity recognition (identifikasi entitas penting), hingga pemahaman semantik yang lebih kompleks menggunakan model bahasa berbasis transformer.

Dalam sistem klasifikasi aduan, NLP berperan untuk menganalisis teks aduan yang ditulis masyarakat dengan gaya bahasa sehari-hari, termasuk istilah lokal atau singkatan informal, kemudian mengambil informasi penting seperti jenis permasalahan, lokasi kejadian, dan tingkat urgensi. Teknologi ini menjadi fondasi bagi model AI untuk memahami konteks aduan tanpa perlu format input yang kaku atau terstruktur.

2.2.16 Large Language Model (LLM)

Large Language Model (LLM) adalah model kecerdasan buatan berbasis arsitektur model AI modern yang dilatih pada kumpulan teks yang sangat banyak untuk memahami dan menghasilkan bahasa natural (Vaswani et al., 2017). Berbeda dengan model metode machine learning lama yang perlu dilatih ulang untuk setiap tugas spesifik, LLM memiliki kemampuan menghasilkan yang memungkinkan model untuk menyelesaikan berbagai tugas bahasa hanya dengan pemberian instruksi atau prompt yang tepat, tanpa perlu data latihan tambahan yang banyak.

Dalam sistem pengelolaan aduan, LLM digunakan untuk

mengklasifikasikan teks aduan ke dalam kategori, sub-kategori, dan OPD tujuan secara otomatis. Model ini mampu memahami konteks aduan meskipun ditulis dengan bahasa informal atau menggunakan istilah lokal Ketapang, karena telah dilatih pada kumpulan teks yang sangat banyak bahasa yang sangat luas. Pendekatan ini jauh lebih efisien dibanding metode konvensional yang memerlukan ratusan atau ribuan data yang sudah dikategorikan untuk setiap kategori baru.

2.2.17 Prompt Engineering

Prompt Engineering adalah teknik merancang dan mengoptimalkan instruksi teks (prompt) yang diberikan kepada model bahasa besar (Large Language Model/LLM) untuk mendapatkan output yang akurat dan relevan sesuai kebutuhan (White et al., 2023). Dalam konteks penggunaan LLM untuk klasifikasi teks, kualitas prompt secara langsung mempengaruhi akurasi dan konsistensi hasil klasifikasi. Prompt engineering mencakup pemilihan kata kunci yang tepat, penyusunan struktur instruksi yang jelas, pemberian contoh (few-shot examples), dan penentuan format output yang diinginkan.

Terdapat beberapa teknik prompt engineering yang umum digunakan. Zero-shot prompting adalah teknik memberikan instruksi tanpa contoh, mengandalkan kemampuan model untuk memahami tugas dari deskripsi saja. Few-shot prompting adalah teknik memberikan beberapa contoh input-output sebelum meminta model melakukan tugas yang sebenarnya, sehingga model dapat belajar pola dari contoh tersebut. Chain-of-thought prompting adalah teknik yang meminta model untuk menjelaskan cara berpikir atau langkah-langkah sebelum memberikan jawaban akhir, meningkatkan akurasi untuk tugas yang kompleks. System prompt adalah instruksi awal yang mendefinisikan peran dan batasan model, seperti "Anda adalah sistem klasifikasi aduan yang harus selalu menghasilkan output dalam format JSON."

Dalam penerapan untuk sistem klasifikasi teks, prompt engineering dapat ditingkatkan melalui beberapa pendekatan. Pertama, pemberian contoh konkret (few-shot examples) yang menunjukkan pola klasifikasi yang benar membantu model memahami ekspektasi output. Kedua, injeksi pengetahuan domain (domain knowledge injection) berupa daftar kategori, sub-kategori, dan aturan klasifikasi dapat meningkatkan akurasi terutama untuk konteks yang spesifik. Ketiga,

penyediaan konteks tambahan seperti data historis atau informasi terkait dapat membantu model membuat keputusan yang lebih baik, misalnya dalam mendeteksi duplikasi dengan membandingkan teks baru terhadap data sebelumnya.

Proses pengembangan prompt yang efektif umumnya bersifat iteratif, melibatkan evaluasi hasil pada data uji, identifikasi pola kesalahan, dan perbaikan prompt berdasarkan analisis error. Pendekatan prompt engineering yang tepat dapat mencapai performa tinggi tanpa perlu melakukan fine-tuning model yang membutuhkan biaya komputasi dan data latihan yang besar, menjadikannya solusi yang praktis dan efisien untuk berbagai aplikasi klasifikasi teks.

2.2.18 Laravel

Laravel merupakan framework PHP open-source yang dikembangkan oleh Taylor Otwell dan menggunakan pola arsitektur Model-View-Controller (MVC) untuk mempermudah pengembangan aplikasi web secara terstruktur (Harianto dkk., 2019). Laravel menyediakan berbagai fitur bawaan yang mempercepat proses pengembangan, seperti Eloquent ORM untuk pengelolaan database secara object-oriented, Blade sebagai template engine untuk tampilan antarmuka, serta Artisan CLI untuk otomatisasi tugas-tugas pengembangan seperti membuat model, controller, dan migration.

Framework ini juga dilengkapi dengan sistem middleware untuk autentikasi dan otorisasi, migration untuk pengelolaan struktur database secara memiliki versi, serta cara kerja antrian tugas untuk menangani proses berat di background secara tidak sinkron. Ekosistem Laravel yang kaya, termasuk sistem autentikasi API Laravel untuk autentikasi API dan sistem monitoring antrian Laravel untuk monitoring queue, ditambah dengan komunitas pengembang yang besar serta dokumentasi yang lengkap, menjadikan Laravel salah satu framework PHP paling populer untuk pengembangan aplikasi web modern.

2.2.19 PHP (Hypertext Preprocessor)

PHP adalah bahasa pemrograman server-side yang banyak digunakan untuk pengembangan aplikasi web dinamis. PHP pertama kali dikembangkan oleh Rasmus Lerdorf pada tahun 1994 dan terus berkembang hingga menjadi salah satu bahasa pemrograman web paling populer di dunia (PHP Group, 2023). PHP bersifat open-source sehingga dapat digunakan secara gratis, mendukung berbagai sistem

manajemen basis data seperti MySQL, PostgreSQL, dan SQLite, serta memiliki komunitas pengembang yang besar dengan dokumentasi yang lengkap.

PHP mendukung cara pemrograman berorientasi objek secara penuh, termasuk konsep-konsep pemrograman objek. Hal ini memungkinkan pengembang untuk menulis kode yang modular, terstruktur, dan mudah dikelola dalam skala proyek yang besar.

2.2.20 MySQL

MySQL adalah Sistem Manajemen Basis Data Relasional open-source yang menggunakan Structured Query Language (SQL) untuk mengelola dan memanipulasi data (Lim & Silalahi, 2023). MySQL menjadi pilihan yang tepat karena mampu menangani aspek keamanan seperti pengaturan nama host, hak akses, dan kata sandi. Selain itu, MySQL memiliki performa tinggi dalam menangani operasi baca dan tulis data dalam jumlah besar (Jubilee Enterprise, 2018 dalam Lim & Silalahi, 2023).

MySQL mendukung berbagai fitur penting untuk pengembangan aplikasi modern, seperti transaction dan ACID compliance untuk menjaga integritas data, pengindeksan untuk mempercepat proses pencarian, batasan kunci asing untuk konsistensi relasi antar tabel, prosedur tersimpan dan pemicu untuk logika bisnis di level database, serta replication dan backup untuk keamanan data.

2.2.21 Arsitektur Model-View-Controller (MVC)

Model-View-Controller (MVC) adalah pola arsitektur perangkat lunak yang membagi struktur aplikasi menjadi tiga komponen utama yang saling terpisah namun saling terhubung (Rosa & Shalahuddin, 2018). Pemisahan ini bertujuan untuk meningkatkan modularitas, memudahkan pengembangan secara paralel, dan mempermudah pemeliharaan kode di masa mendatang.

Komponen Model bertanggung jawab terhadap pengelolaan data dan logika bisnis aplikasi, berinteraksi langsung dengan database untuk melakukan operasi seperti penyimpanan, pengambilan, pembaruan, dan penghapusan data. Pada framework Laravel, model diimplementasikan menggunakan Eloquent ORM yang memetakan setiap tabel database menjadi sebuah class PHP. Komponen View bertanggung jawab terhadap tampilan antarmuka pengguna, menerima data dari controller dan menyajikannya dalam format yang dapat dibaca

melalui browser—pada Laravel dikelola menggunakan Blade Template Engine. Sedangkan Controller berperan sebagai perantara yang menerima input dari pengguna melalui request HTTP, memproses logika bisnis dengan memanggil model yang sesuai, dan mengembalikan respons dalam bentuk view atau data.

Arsitektur MVC memberikan separation of concerns yang jelas sehingga setiap komponen dapat dikembangkan dan diuji secara independen, memudahkan pengembangan secara tim karena masing-masing pengembang dapat bekerja pada komponen berbeda tanpa saling mengganggu, serta mempermudah pemeliharaan dan pengembangan fitur baru tanpa harus mengubah keseluruhan sistem.

2.2.22 Pemrograman Berorientasi Objek (PBO)

Pemrograman Berorientasi Objek (PBO) atau Object-Oriented Programming (OOP) adalah cara pemrograman yang mengorganisasikan perangkat lunak ke dalam objek-objek yang terdiri atas data (atribut) dan perilaku (method) (Rosa & Shalahuddin, 2018). PBO menjadi cara pemrograman yang dominan dalam pengembangan perangkat lunak modern karena mampu menggambarkan objek dunia nyata ke dalam struktur kode yang lebih mudah dipahami dan dikelola.

PBO memiliki empat pilar utama. Encapsulation (Enkapsulasi) adalah konsep menyembunyikan detail implementasi internal suatu objek dari dunia luar, sehingga objek hanya dapat diakses melalui antarmuka yang telah ditentukan dan melindungi data dari modifikasi yang tidak diinginkan. Inheritance (Pewarisan) adalah cara kerja yang memungkinkan suatu class untuk mewarisi atribut dan method dari class lain, sehingga mengurangi duplikasi kode dan mendukung penggunaan ulang komponen. Polymorphism (Polimorfisme) adalah kemampuan suatu objek untuk mengambil banyak bentuk, di mana method yang sama dapat memiliki perilaku yang berbeda tergantung pada class yang mengimplementasikannya. Abstraction (Abstraksi) adalah proses penyederhanaan kompleksitas dengan menampilkan hanya informasi yang relevan dan menyembunyikan detail yang tidak perlu, dapat diimplementasikan melalui abstract class atau interface.

Pada bahasa PHP dan framework Laravel, konsep-konsep pemrograman objek diterapkan secara luas—setiap model Eloquent merupakan class yang mewarisi class Model dasar, controller merupakan class yang menangani request

HTTP, dan middleware merupakan class yang memproses request sebelum mencapai controller.

2.2.23 Blade Template Engine

Blade adalah template engine bawaan Laravel yang menyediakan sintaks sederhana namun powerful untuk membuat tampilan antarmuka pengguna (Laravel Documentation, 2023). Blade menggunakan sintaks khusus seperti `{{ }}` untuk menampilkan data dan `@directive` untuk struktur kontrol. Blade mendukung template inheritance sehingga layout dapat digunakan ulang, menyediakan komponen dan slot untuk membuat elemen antarmuka yang reusable, serta melakukan pengamanan otomatis pada output untuk mencegah serangan XSS. Blade juga terintegrasi langsung dengan framework Laravel tanpa konfigurasi tambahan, serta mendukung perintah kondisional seperti `@if`, `@foreach`, `@switch`, dan perintah khusus yang memungkinkan pengembang mendefinisikan sintaks template sesuai kebutuhan proyek.

2.2.24 Livewire

Livewire adalah framework full-stack untuk Laravel yang memungkinkan pembuatan antarmuka dinamis tanpa perlu menulis JavaScript secara langsung (Livewire Documentation, 2023). Livewire menggunakan pendekatan server-side rendering dengan pembaruan real-time melalui AJAX (teknologi update tanpa reload halaman), sehingga pengembang dapat membangun komponen interaktif menggunakan PHP. Keunggulan utama Livewire adalah tidak memerlukan API backend terpisah karena berjalan di atas Laravel, sehingga sangat cocok untuk membangun komponen interaktif pada dashboard seperti notifikasi, pembaruan status, dan tabel data yang perlu di-refresh secara berkala tanpa memerlukan pengetahuan mendalam tentang JavaScript atau framework frontend seperti React atau Vue.

2.2.25 Tailwind CSS

Tailwind CSS adalah framework CSS berbasis utility-first yang menyediakan kelas-kelas atomik untuk styling tampilan web (Tailwind Documentation, 2023). Berbeda dengan framework CSS lainnya seperti Bootstrap yang menyediakan komponen siap pakai (button, card, navbar), Tailwind memberikan building blocks

berupa kelas utilitas yang dapat dikombinasikan untuk membuat desain kustom. Pendekatan utility-first memungkinkan styling langsung pada elemen HTML tanpa perlu menulis file CSS terpisah, sehingga pengembang dapat melihat langsung efek styling pada markup.

Tailwind mendukung desain responsif dengan breakpoint bawaan untuk berbagai ukuran layar (mobile, tablet, desktop), dan menggunakan teknik tree-shaking yang memastikan file CSS akhir hanya berisi kelas yang benar-benar digunakan dalam proyek, sehingga dapat menghasilkan ukuran file yang sangat kecil di produksi. Framework ini juga mudah dikustomisasi melalui file konfigurasi `tailwind.config.js`, memungkinkan pengembang untuk menyesuaikan palet warna, spacing, dan breakpoint sesuai kebutuhan desain.

2.2.26 Laragon

Lara Laragon adalah lingkungan pengembangan web lokal (local development environment) yang ringan, cepat, dan mudah digunakan untuk mengembangkan aplikasi berbasis PHP di sistem operasi Windows (Laragon, 2023). Berbeda dengan XAMPP yang memerlukan konfigurasi manual untuk setiap proyek, atau Docker yang memerlukan pemahaman tentang containerization, Laragon menyediakan stack lengkap (Apache/Nginx, MySQL/MariaDB, PHP, Node.js) dengan sistem auto-detect yang membuat instalasi dan konfigurasi menjadi sangat sederhana.

Laragon menawarkan fitur Pretty URLs yang secara otomatis membuat virtual host untuk setiap proyek, sehingga pengembang tidak perlu konfigurasi manual. Framework ini juga mendukung multiple versi PHP sehingga pengembang dapat beralih antar versi dengan mudah, dan menyediakan terminal bawaan yang sudah dilengkapi dengan Git, Composer, dan Node.js. Dibandingkan dengan Docker yang menggunakan virtual machine dan memerlukan resource lebih besar, Laragon berjalan native di Windows sehingga lebih ringan dan cepat, serta dapat auto-create database saat membuat proyek baru.

2.2.27 Visual Studio Code

Visual Studio Code (VS Code) adalah code editor yang dikembangkan oleh Microsoft, bersifat open-source, ringan, dan mendukung berbagai bahasa pemrograman (Microsoft, 2023). VS Code telah menjadi salah satu code editor paling populer di kalangan pengembang perangkat lunak karena fleksibilitas dan

ekosistem ekstensi yang sangat kaya.

Fitur IntelliSense menyediakan auto-completion, parameter hints, dan definisi hover yang mempercepat penulisan kode, misalnya saat mengetik `$request->`, VS Code langsung menampilkan daftar method yang tersedia di class Request Laravel. Editor ini juga menyediakan integrated terminal yang memungkinkan pengembang menjalankan perintah seperti `php artisan migrate` atau `npm run dev` tanpa keluar dari editor, serta mendukung kontrol versi Git secara bawaan sehingga pengembang dapat melakukan commit, pull, dan push langsung dari sidebar.

Sistem ekstensi VS Code sangat kaya, termasuk ekstensi khusus untuk PHP, Laravel Blade, Tailwind CSS IntelliSense, dan Livewire yang membantu proses pengembangan dengan menyediakan syntax highlighting, auto-completion, dan error detection spesifik untuk setiap teknologi. VS Code juga menyediakan fitur debugging bawaan dengan breakpoint, call stack, dan variabel watch, menjadikannya ringan namun memiliki banyak fitur layaknya Integrated Development Environment (IDE) penuh.

2.2.28 Database

Database atau basis data adalah kumpulan data yang terorganisir secara sistematis dan disimpan dalam media penyimpanan elektronik sehingga dapat diakses, dikelola, dan diperbarui secara efisien (Rosa & Shalahuddin, 2018). Dalam pengembangan aplikasi web modern, database menjadi komponen fundamental yang menyimpan seluruh informasi yang dibutuhkan oleh aplikasi.

Database relasional (Relational Database Management System/RDBMS) merupakan jenis database yang paling banyak digunakan, di mana data disimpan dalam bentuk tabel-tabel yang saling berelasi (Lim & Silalahi, 2023). Konsep utama database relasional meliputi:

1. Tabel merupakan struktur dasar penyimpanan data yang terdiri atas baris (record) dan kolom (field).
2. Primary Key merupakan kolom unik yang mengidentifikasi setiap baris dalam tabel secara unik.
3. Foreign Key merupakan kolom yang merujuk ke primary key di tabel lain, membentuk relasi antar tabel.

4. Index merupakan struktur data tambahan yang mempercepat pencarian data dalam tabel yang berukuran besar.
5. Normalisasi merupakan proses perancangan struktur tabel untuk mengurangi redundansi data dan menjaga integritas data.

Perancangan database yang baik sangat menentukan performa, skalabilitas, dan keandalan suatu aplikasi. Oleh karena itu, tahap perancangan database perlu dilakukan dengan cermat, mulai dari identifikasi entitas, penentuan atribut, hingga penetapan relasi antar tabel.

2.2.29 Google AI Studio

Google AI Studio adalah platform pengembangan AI dari Google yang menyediakan akses ke berbagai model Large Language Model melalui REST API (Google AI Documentation, 2024). Platform ini memungkinkan pengembang untuk mengintegrasikan kemampuan AI ke dalam aplikasi melalui API key, dengan menyediakan berbagai model AI dengan karakteristik yang berbeda, mulai dari model ringan untuk tugas sederhana hingga model besar seperti Gemma untuk tugas kompleks yang memerlukan pemahaman konteks mendalam.

Google AI Studio menyediakan kuota gratis yang memadai untuk pengembangan dan pengujian, dilengkapi dashboard untuk monitoring penggunaan API yang membantu pengembang melacak konsumsi tokens dan performa model. Platform ini mendukung berbagai format output termasuk JSON terstruktur yang memudahkan integrasi dengan sistem backend, serta mendukung format prompt yang fleksibel, termasuk system prompt untuk mengatur perilaku model dan user prompt untuk memberikan instruksi spesifik, sehingga pengembang dapat mengkustomisasi respons model sesuai dengan kebutuhan aplikasi.

2.2.30 Gemma 4 31B IT

Gemma 4 31B IT adalah Large Language Model open-weight dari Google dengan 31 miliar parameter, merupakan generasi ke-4 dari keluarga Gemma yang dirancang untuk tugas instruction-following dengan dukungan bahasa non-Inggris termasuk bahasa Indonesia (Google Research, 2026). Model ini dilatih pada dataset multilingual yang luas sehingga mampu memahami konteks bahasa dan dialek lokal, termasuk istilah informal seperti "aik dak jalan" (air tidak mengalir dalam

bahasa daerah Ketapang) atau variasi penulisan yang umum di media sosial.

Dengan 31 miliar parameter, Gemma 4 31B IT mampu memahami konteks kalimat yang kompleks dan nuansa bahasa yang tidak dapat ditangani oleh model yang lebih kecil. Model ini tersedia pada kuota gratis Google AI Studio dengan gratis tanpa batasan penggunaan per menit, menjadikannya pilihan ekonomis untuk pengembangan sistem klasifikasi. Berbeda dengan model AI berbayar yang memerlukan biaya per request, Gemma bersifat open-weight sehingga dapat memudahkan pengembang untuk memahami cara kerja model dan mengoptimalkan prompt tanpa bergantung pada sistem tertutup yang tidak dapat diperiksa.

Model ini termasuk dalam kategori instruction-tuned, artinya sudah dilatih khusus untuk mengikuti instruksi yang diberikan dalam bentuk prompt, sehingga sangat cocok untuk tugas-tugas seperti klasifikasi teks, pengambilan informasi, dan analisis konten yang memerlukan pemahaman mendalam terhadap bahasa Indonesia.

2.2.31 API (Application Programming Interface)

API (Application Programming Interface) adalah antarmuka yang memungkinkan dua aplikasi atau sistem untuk berkomunikasi dan bertukar data secara terprogram (Satzinger et al., 2019). API menyediakan seperangkat aturan dan protokol yang menentukan bagaimana permintaan data harus dibuat dan bagaimana respons akan dikembalikan, sehingga memungkinkan integrasi antar sistem tanpa perlu mengetahui detail implementasi internal masing-masing sistem.

REST API (Representational State Transfer) adalah gaya arsitektur API yang paling umum digunakan, di mana komunikasi dilakukan melalui protokol HTTP dengan metode standar seperti GET (mengambil data), POST (mengirim data baru), PUT (memperbarui data), dan DELETE (menghapus data). REST API menggunakan format data yang ringan seperti JSON atau XML untuk pertukaran informasi, membuatnya mudah dibaca baik oleh mesin maupun manusia.

Dalam konteks integrasi dengan layanan eksternal, API berperan sebagai jembatan komunikasi yang memungkinkan aplikasi lokal mengakses fungsionalitas dari layanan cloud seperti model kecerdasan buatan, layanan penyimpanan, atau sistem pihak ketiga lainnya. Komunikasi melalui API umumnya menggunakan mekanisme autentikasi seperti API key atau token yang disimpan secara aman di

file konfigurasi, memastikan hanya aplikasi yang terotorisasi yang dapat mengakses layanan tersebut.

2.2.32 Playwright

Playwright adalah framework automation browser yang dikembangkan oleh Microsoft untuk mendukung pengujian dan otomatisasi interaksi pada aplikasi web (Microsoft Documentation, 2023). Playwright mendukung berbagai browser seperti Chromium, Firefox, dan WebKit dengan satu API yang konsisten, memungkinkan pengembang untuk menulis skrip otomatis yang berjalan di berbagai browser tanpa perlu mengubah kode.

Sebagai contoh, untuk mengambil data aduan dari halaman Facebook, Playwright dapat membuka browser, login menggunakan sesi yang tersimpan (sehingga tidak perlu login ulang setiap kali), scroll halaman untuk memuat konten dinamis yang di-load via JavaScript, lalu mengekstrak teks postingan dan komentar. Framework ini memiliki mekanisme auto-wait yang secara otomatis menunggu elemen siap sebelum melakukan aksi, mengurangi kegagalan akibat elemen belum dimuat, berbeda dengan HTTP request biasa yang hanya mengambil HTML statis tanpa menjalankan JavaScript.

Playwright juga mampu melakukan intersepsi jaringan untuk memblokir request ke iklan atau resource yang tidak perlu, mengoptimalkan kecepatan scraping. Mode headless memungkinkan scraping berjalan tanpa membuka jendela browser (untuk otomatisasi server), sedangkan mode headful berguna untuk debugging dengan melihat langsung apa yang dilakukan browser.

2.2.33 Node.js

Node.js adalah runtime environment JavaScript yang berjalan di sisi server, dibangun di atas V8 JavaScript engine dari Google Chrome (Node.js Foundation, 2023). Node.js memungkinkan eksekusi kode JavaScript di luar browser dengan arsitektur yang bersifat asynchronous dan tidak memblokir proses input/output, sehingga sangat efisien untuk tugas-tugas yang melibatkan operasi input/output intensif seperti pengambilan data dari web, pembacaan file, atau koneksi database.

Arsitektur asynchronous Node.js memungkinkan server menangani ribuan koneksi secara bersamaan tanpa membuat thread terpisah untuk setiap koneksi, berbeda dengan model multithreading tradisional yang memerlukan resource lebih

besar. Node.js juga memiliki ekosistem package manager (NPM) terbesar di dunia dengan jutaan paket yang tersedia, memungkinkan pengembang untuk menggunakan library siap pakai seperti Playwright untuk web scraping, Express untuk web server, atau Socket.io untuk komunikasi real-time.

Dalam konteks scraping, Node.js sering digunakan sebagai runtime untuk menjalankan tools seperti Playwright atau Puppeteer yang memerlukan eksekusi JavaScript di sisi server, mendukung pengembangan full-stack dengan satu bahasa pemrograman, serta ringan dan cepat dalam menangani operasi jaringan dan file system.

2.2.34 Authentication dan Authorization

Authentication dan Authorization adalah dua konsep keamanan fundamental dalam sistem informasi yang saling melengkapi (Stallings, 2019). **Authentication (autentikasi)** adalah proses memverifikasi identitas pengguna yang mencoba mengakses sistem, biasanya melalui kombinasi username dan password, token, atau metode biometrik. Sedangkan **Authorization (otorisasi)** adalah proses menentukan hak akses pengguna yang telah terautentikasi—menentukan data atau fitur apa saja yang boleh diakses oleh pengguna tersebut.

Proses authentication umumnya melibatkan tiga faktor: something you know (sesuatu yang Anda ketahui, seperti password), something you have (sesuatu yang Anda miliki, seperti token atau kartu akses), dan something you are (sesuatu yang Anda miliki secara biologis, seperti sidik jari). Dalam konteks aplikasi web, authentication paling umum menggunakan kombinasi email/username dan password yang di-hash menggunakan algoritma aman seperti bcrypt atau Argon2 untuk melindungi credential pengguna.

Authorization bekerja setelah proses authentication berhasil. Sistem akan memeriksa peran (role) atau permission yang dimiliki pengguna untuk menentukan akses yang diizinkan. Dalam framework web modern seperti Laravel, authorization dapat diimplementasikan menggunakan middleware, gates, dan policies—middleware memastikan hanya pengguna terautentikasi yang dapat mengakses resource tertentu, sementara gates dan policies mengatur aturan akses yang lebih granular berdasarkan peran atau kondisi spesifik. Implementasi yang tepat dari kedua konsep ini sangat penting untuk menjaga keamanan sistem dan memastikan

bahwa data sensitif hanya dapat diakses oleh pihak yang berwenang.

2.2.35 Role-Based Access Control (RBAC)

Role-Based Access Control (RBAC) adalah model keamanan yang mengatur hak akses pengguna terhadap sumber daya sistem berdasarkan peran yang dimiliki, sehingga setiap pengguna hanya dapat mengakses fungsi dan data sesuai dengan tanggung jawabnya (Hidayati & Rochimah, 2019). RBAC terdiri dari tiga komponen utama yaitu pengguna, peran, dan hak akses, setiap pengguna diberikan satu atau lebih peran, dan setiap peran memiliki sekumpulan hak akses tertentu.

Dalam sistem pengelolaan aduan KMC, RBAC memastikan bahwa admin dapat melihat dan mengelola semua aduan, petugas OPD hanya dapat melihat aduan yang ditugaskan ke instansinya, dan operator hanya memiliki akses terbatas untuk input data tanpa hak menghapus. Model ini memudahkan pengelolaan hak akses karena cukup mengubah peran pengguna ketika ada perubahan tanggung jawab, mendukung prinsip least privilege di mana setiap pengguna hanya mendapatkan hak akses minimum yang dibutuhkan, serta mudah diaudit karena setiap aksi dapat dilacak berdasarkan peran yang dimiliki.

2.2.36 Black Box Testing

Black Box Testing adalah metode pengujian perangkat lunak yang berfokus pada fungsionalitas sistem tanpa mengetahui struktur internal atau kode program yang diuji (Mustaqbal et al., 2015). Pengujian ini berfokus pada input dan output sistem, yaitu memastikan bahwa fungsi-fungsi yang tersedia berjalan sesuai dengan kebutuhan yang telah ditentukan. Black Box Testing tidak memerlukan pengetahuan tentang kode program sehingga dapat dilakukan oleh penguji yang bukan programmer, dilakukan dari sudut pandang pengguna akhir sehingga lebih relevan dengan pengalaman pengguna nyata, serta efektif untuk menemukan kesalahan pada fungsi, antarmuka, dan validasi data dari berbagai tipe input termasuk input yang valid, tidak valid, dan ekstrem.

Salah satu teknik dalam Black Box Testing adalah Boundary Value Analysis (BVA), yaitu pengujian yang difokuskan pada nilai-nilai batas dari rentang input yang valid. Teknik ini efektif karena sebagian besar kesalahan pada perangkat lunak sering terjadi pada nilai-nilai batas, bukan pada nilai tengah.

2.2.37 Unified Modelling Language (UML)

Unified Modelling Language (UML) adalah bahasa pemodelan visual yang digunakan untuk menspesifikasikan, merancang, dan mendokumentasikan sistem perangkat lunak (Rosa & Shalahuddin, 2018). UML menyediakan berbagai jenis diagram yang dapat digunakan untuk memodelkan sistem dari berbagai sudut pandang, baik struktural maupun perilaku. Penggunaan UML dalam pengembangan sistem membantu pengembang dan pemangku kepentingan untuk memiliki pemahaman yang sama mengenai desain sistem sebelum tahap implementasi dimulai.

UML terdiri dari dua kelompok diagram utama, yaitu Diagram Struktural yang menggambarkan struktur statis sistem seperti Class Diagram, Component Diagram, dan Deployment Diagram, serta Diagram Perilaku yang menggambarkan perilaku dinamis sistem seperti Use Case Diagram, Activity Diagram, dan Sequence Diagram.

2.2.38 Use Case Diagram

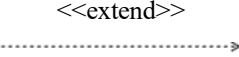
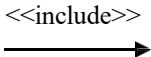
Use Case Diagram adalah diagram yang menggambarkan interaksi antara aktor dengan sistem serta fungsionalitas yang disediakan, menunjukkan bagaimana pengguna berinteraksi dengan sistem untuk mencapai tujuan tertentu (Satzinger et al., 2019). Use case diagram menunjukkan hubungan antara aktor dan use case dalam suatu sistem. Terdapat dua komponen utama dalam use case diagram:

1. Use Case merupakan fungsionalitas yang disediakan sistem sebagai unit-unit yang saling bertukar pesan antar unit atau aktor.
2. Aktor merupakan orang, proses, atau sistem lain yang berinteraksi dengan sistem informasi yang akan dibuat di luar sistem informasi itu sendiri.

Adapun simbol-simbol use case diagram dapat dilihat pada Tabel 2.1.

Tabel 2.1.Simbol *Use Case Diagram*.

Simbol	Nama	Keterangan
	<i>Use Case</i>	Menggambarkan suatu perilaku dari sistem tanpa mengungkapkan struktur internal dari sistem tersebut.
	<i>Actor</i>	Digunakan untuk menjelaskan sesuatu atau seseorang yang sedang berinteraksi dengan sistem

	<i>Association</i>	Jalur komunikasi antar <i>actor</i> dengan <i>use case</i> yang saling berpartisipasi
	<i>Extend</i>	Penambahan perilaku ke dalam <i>use case</i> dasar yang tidak tahu tentang hal tersebut tambahan memiliki nama depan yang sama dengan <i>use case</i> yang ditambahkan.
	<i>Generalization</i>	Hubungan antara <i>use case</i> umum dengan <i>use case</i> yang lebih spesifik, yang mewarisi dan menambah fitur terhadapnya
	<i>Include</i>	Penambahan perilaku ke dalam <i>use case</i> dasar yang secara eksplisit menjelaskan penambahannya.

Sumber : Susilowati & Pakusadewa, 2023

Setiap use case dalam diagram dijelaskan lebih rinci melalui use case scenario, yaitu deskripsi langkah-langkah interaksi antara aktor dan sistem dalam menjalankan suatu use case. Use case scenario memuat informasi seperti nama use case, aktor yang terlibat, deskripsi singkat, pre-condition, post-condition, serta alur utama dan alur alternatif dari interaksi tersebut (Nugroho, 2019).

2.2.39 Activity Diagram

Activity Diagram merupakan diagram yang menggambarkan alur kerja (workflow) atau aktivitas dari sebuah sistem atau proses bisnis (Harianto dkk., 2019). Activity diagram menunjukkan urutan aktivitas dalam suatu proses, termasuk aktivitas yang dilakukan secara berurutan maupun bersamaan, serta keputusan (decision) yang mungkin terjadi dalam alur tersebut. Activity diagram berguna untuk memodelkan proses bisnis yang kompleks dan membantu memahami alur logika suatu fitur sebelum diimplementasikan dalam kode program. Adapun simbol-simbol activity diagram dapat dilihat pada Tabel 2.2.

Tabel 2.2 Simbol *Activity Diagram*

Simbol	Nama	Keterangan
	<i>Start</i>	Menunjukkan di mana aliran kerja dimulai.
	<i>Final</i>	Menunjukkan di mana aliran kerja berakhir.
	<i>Action</i>	Langkah-langkah dalam sebuah <i>activity</i> .

	<i>Decision</i>	Menunjukkan di mana keputusan akan dibuat.
	<i>Swimlane</i>	Mengelompokkan <i>activity</i> berdasarkan <i>actor</i> .

Sumber : Susilowati & Pakusadewa, 2023

2.2.40 Sequence Diagram

Sequence Diagram adalah diagram yang menggambarkan interaksi antar objek dalam suatu sistem berdasarkan urutan waktu (Herlinah & Musliadi, 2019). Sequence diagram menunjukkan bagaimana pesan dikirim dan diterima antar objek untuk melaksanakan suatu skenario tertentu. Diagram ini berguna untuk memahami alur komunikasi antara komponen-komponen sistem, mulai dari antarmuka pengguna (boundary), logika bisnis (controller), hingga penyimpanan data (entity). Adapun simbol-simbol sequence diagram dapat dilihat pada Tabel 2.3.

Tabel 2.5 Simbol-simbol *Sequence Diagram*

Simbol	Nama	Keterangan
	<i>Entity Class</i>	Gambaran sistem sebagai landasan dalam menyusun basis data
	<i>Boundary Class</i>	Menangani komunikasi antar lingkungan sistem
	<i>Control Class</i>	Bertanggung jawab terhadap objek yang berisi logika
	<i>Recursive</i>	Pesan untuk dirinya
	<i>Activation</i>	Mewakili proses durasi aktivasi sebuah operasi

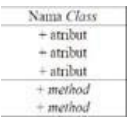
Sumber : Susilowati & Pakusadewa, 2023

2.2.41 Class Diagram

Class Diagram adalah diagram yang menggambarkan struktur statis dari sistem dengan menunjukkan kelas-kelas yang ada, atribut dan metode yang dimiliki oleh masing-masing kelas, serta hubungan antar kelas tersebut (Puspitasari & Setyawan, 2020). Class diagram merupakan salah satu diagram UML yang paling penting

dalam pemodelan berorientasi objek karena menjadi dasar bagi implementasi kode program. Perancangan class diagram yang baik memastikan bahwa struktur kode program mencerminkan arsitektur sistem yang telah direncanakan, sehingga memudahkan proses pengembangan dan pemeliharaan. Adapun simbol-simbol class diagram dapat dilihat pada Tabel 2.4.

Tabel 2.4 Simbol-simbol *Class Diagram*

Simbol	Nama	Keterangan
	<i>Class</i>	Himpunan objek-objek dari berbagai atribut yang memiliki operasi yang sama.
	<i>Directed Association</i>	Relasi antar kelas dengan makna kelas yang satu digunakan oleh kelas lain.
	<i>Aggregation</i>	Mengindikasikan keseluruhan bagian <i>relationship</i> disebut sebagai relasi.
	<i>Dependency</i>	Menunjukkan operasi pada suatu <i>class</i> yang menggunakan <i>class</i> yang lain.
	<i>Composition</i>	Relasi <i>Composition</i> terhadap <i>class</i> tempat dia bergantung.

Sumber : Susilowati & Pakusadewa, 2023

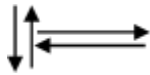
2.2.42 Flowchart

Flowchart adalah representasi grafis dari langkah-langkah dan urutan prosedur dalam sebuah sistem atau proses, yang menggambarkan alur logika secara visual menggunakan simbol-simbol standar (Sutanti et al., 2020). Flowchart membantu analis dan programmer untuk memecah permasalahan menjadi bagian-bagian yang lebih kecil sehingga lebih mudah dianalisis dan dipahami. Dalam pengembangan sistem informasi, flowchart digunakan pada tahap perancangan untuk memodelkan alur proses bisnis dan alur sistem sebelum diimplementasikan dalam kode program. Penggunaan flowchart memberikan beberapa keuntungan dalam perancangan sistem:

1. Mempermudah komunikasi antara pengembang dan pemangku kepentingan

- karena menggunakan representasi visual yang mudah dipahami.
2. Membantu mengidentifikasi redundansi atau kesalahan logika dalam alur proses sebelum implementasi.
 3. Menjadi dokumentasi yang berguna untuk pemeliharaan sistem di masa mendatang.
 4. Memudahkan evaluasi dan optimalisasi proses dengan melihat gambaran keseluruhan alur kerja sistem.

Adapun simbol-simbol yang digunakan dalam flowchart dapat dilihat pada Tabel 2.5.

No.	Simbol	Nama	Keterangan
1		<i>Processing</i>	Menunjukkan pengolahan yang dilakukan oleh komputer
2		<i>Terminator</i>	Untuk permulaan (<i>start</i>) atau akhir (<i>stop</i>) dari suatu kegiatan
3		<i>Decision</i>	Pemilihan proses berdasarkan kondisi yang ada
4		<i>Input-Output</i>	Menyatakan proses <i>input</i> dan <i>output</i> tanpa tergantung dengan jenis peralatannya.
5		<i>Flow Direction</i>	Digunakan untuk menghubungkan antara simbol yang satu dengan simbol yang lain. Simbol ini juga disebut <i>connecting line</i> .

Sumber : Sutanti dkk., 2020

2.3 Profil Tempat Penelitian

Penelitian Tugas Akhir ini dilaksanakan di Ketapang Media Center (KMC) yang berada di bawah naungan Dinas Komunikasi dan Informatika Kabupaten Ketapang. KMC merupakan bagian yang berperan dalam mengelola informasi dan aduan masyarakat yang masuk melalui media sosial maupun kanal digital lainnya. Dalam pelaksanaan tugasnya, KMC menerima berbagai bentuk laporan masyarakat yang perlu dipantau, diklasifikasikan, dan diteruskan kepada pihak terkait sesuai dengan bidang permasalahannya.

Pada kondisi saat ini, pengelolaan aduan masih banyak dilakukan secara

manual sehingga petugas perlu membuka satu per satu akun atau notifikasi yang masuk. Hal ini berpotensi memperlambat proses pencatatan, klasifikasi, dan tindak lanjut aduan. Oleh karena itu, dibutuhkan sebuah sistem yang dapat membantu KMC mengelola aduan secara lebih terpusat, cepat, dan terdokumentasi dengan baik. Melalui penelitian ini, sistem dikembangkan sebagai solusi untuk membantu petugas KMC dalam memantau aduan dari Facebook dan Instagram, menampilkan notifikasi secara terstruktur, serta memberikan rekomendasi klasifikasi berbantuan AI. Dengan adanya sistem ini, diharapkan proses pengelolaan aduan di lingkungan KMC menjadi lebih efektif, efisien, dan mudah dipantau.